Design and Implementation of Remote Switch

Harshvardhan Gupta

This report documents the complete design, implementation, testing, and deployment of a Remote Switch developed for the inauguration ceremony conducted during the university convocation of the graduating batch of 2025. The event was attended by the Lieutenant Governor of Delhi and the Chief Minister of Delhi, which made system reliability, operational safety, and intentional control critically important. The system was designed and used to remotely trigger and play four predefined video sequences corresponding to the inauguration of two laboratories and two Netaji Subhas Chandra Bose statues located at the East and West campuses of the university.

The system was designed to ensure deliberate activation only, prevent accidental triggering, provide clear real-time operator feedback, and maintain reliable operation under battery-powered conditions. A laptop-based Python application received wireless commands from the hardware unit and initiated video playback.

The electronics were centered around an Arduino microcontroller interfaced with an HC-05 Bluetooth module. The primary actuator was an emergency stop switch, intentionally chosen to enforce deliberate operation. Once pressed, the system required explicit re-arming before accepting any further commands, ensuring fail-safe behavior. A secondary large pushbutton was used to cycle through the available video options. LED indicators provided clear visual feedback of the currently selected video state.

Several system-level indicators and protection mechanisms were implemented. A power-on indicator LED, controlled via an SPDT power switch, illuminated whenever the system was energized. A low battery protection circuit was implemented using a resistive potential divider connected to the Arduino's ADC. The battery voltage was continuously monitored in software using the internal 1.12 V reference of the microcontroller. When the computed battery voltage dropped below 6 V, the firmware automatically disabled all system operations, forced all video outputs LOW, and enabled only the low-battery indicator LED, preventing unreliable operation under undervoltage conditions.

An emergency switch status LED provided unambiguous real-time feedback. When the emergency switch was in the released (armed) state, the LED blinked at 2 Hz. When the switch was pressed, the LED remained continuously ON. This behavior was implemented using Timer1 configured in CTC mode with an interrupt service routine.

The system was powered using a 9 V alkaline battery connected to the Arduino Vin pin. Electrical testing revealed transient current spikes of up to approximately 200 mA during Bluetooth data transmission, which were accounted for during power budgeting and endurance testing.

The circuit was assembled on a zero PCB with careful attention to mechanical stability. Mounting holes were created using self-threading screws aligned with enclosure mounting points. The battery was secured using double-sided tape and connected via a dedicated battery connector. Plastic-insulated single-strand wires were used on the PCB backside for rigidity, while plastic-insulated multi-strand wires were used for external and movable connections. Wire lengths were intentionally kept long to allow enclosure opening without stressing connections.

The large pushbutton was soldered directly onto the zero PCB. To extend it to the top of the enclosure, its cap was glued using Elfy adhesive to a pen body cut to the appropriate length. The other end of the pen body was glued to a separate pushbutton cap, providing a rigid and functional actuator accessible from the enclosure exterior.

The HC-05 Bluetooth module was mounted horizontally using a 90-degree female Berg strip to reduce enclosure height and improve mechanical stability. All components were housed inside an IP65-rated plastic enclosure. LEDs were fixed using hot glue and two-part epoxy. Custom 3D-printed decals were attached using Elfy adhesive for clear labeling and professional finish.

A Python-based application running on a laptop received Bluetooth serial commands from the Arduino and triggered playback of predefined video files. The system underwent extensive validation including wireless range testing, endurance testing, electrical stress testing, and multiple rehearsals. The system performed flawlessly during the live inauguration event.

System Block Diagram

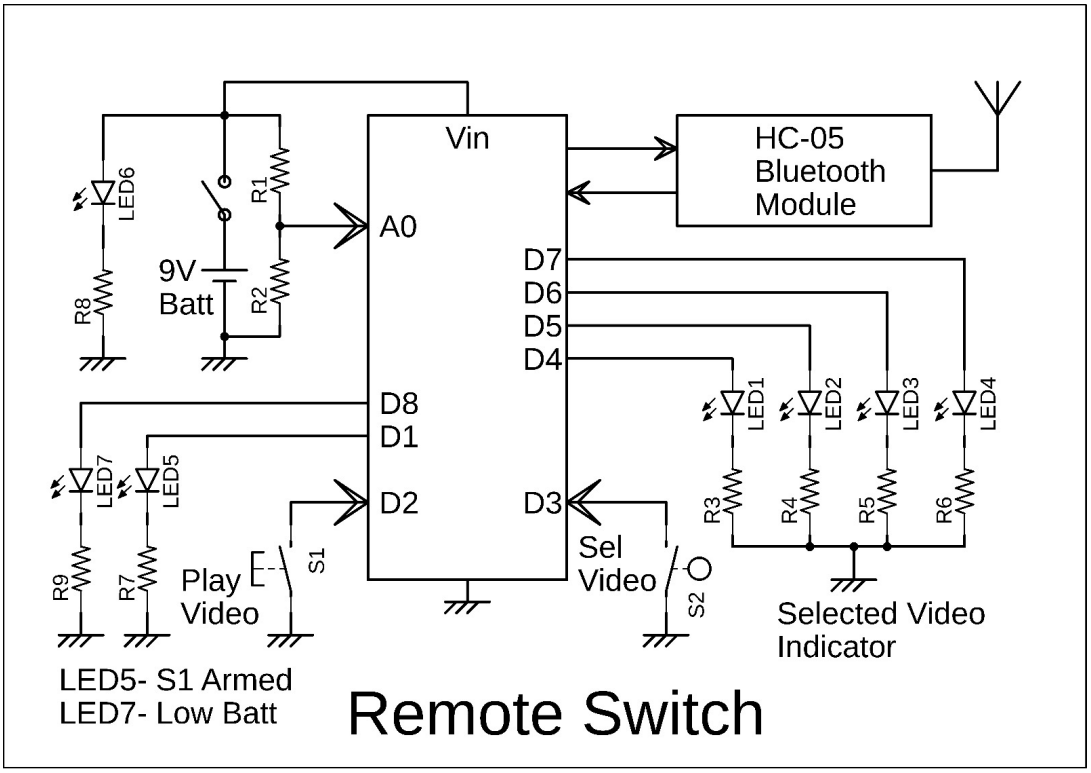


Figure 1: System block diagram of the Remote Switch

Circuit Schematic

Figure 2 shows the complete schematic of the implemented system, including the Arduino, HC-05 Bluetooth module with RX voltage divider (1 kΩ/2 kΩ), emergency switch, pushbutton, LED indicators with 220 Ω current-limiting resistors, battery voltage divider (1 MΩ/120 kΩ), and power input circuitry.

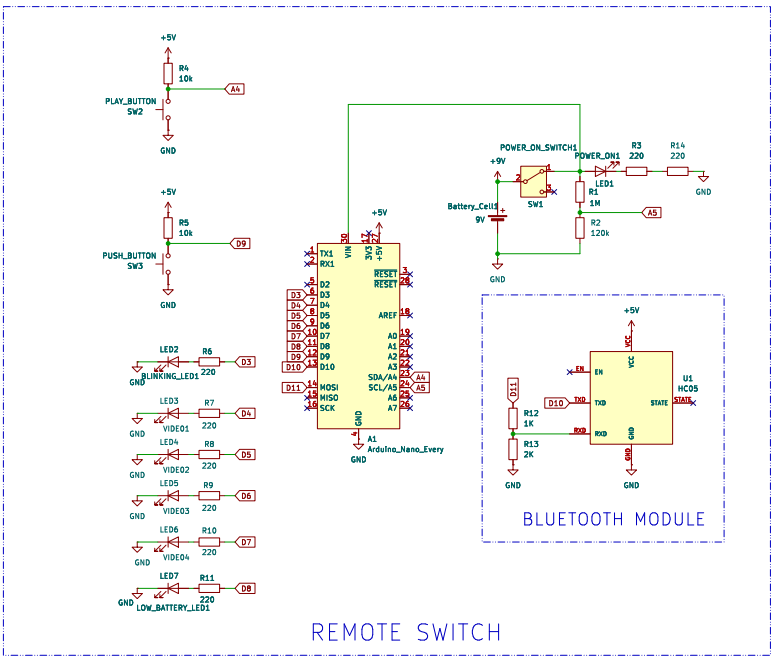


Figure 2: Complete schematic of the Remote Switch

Remote Switch Photographs

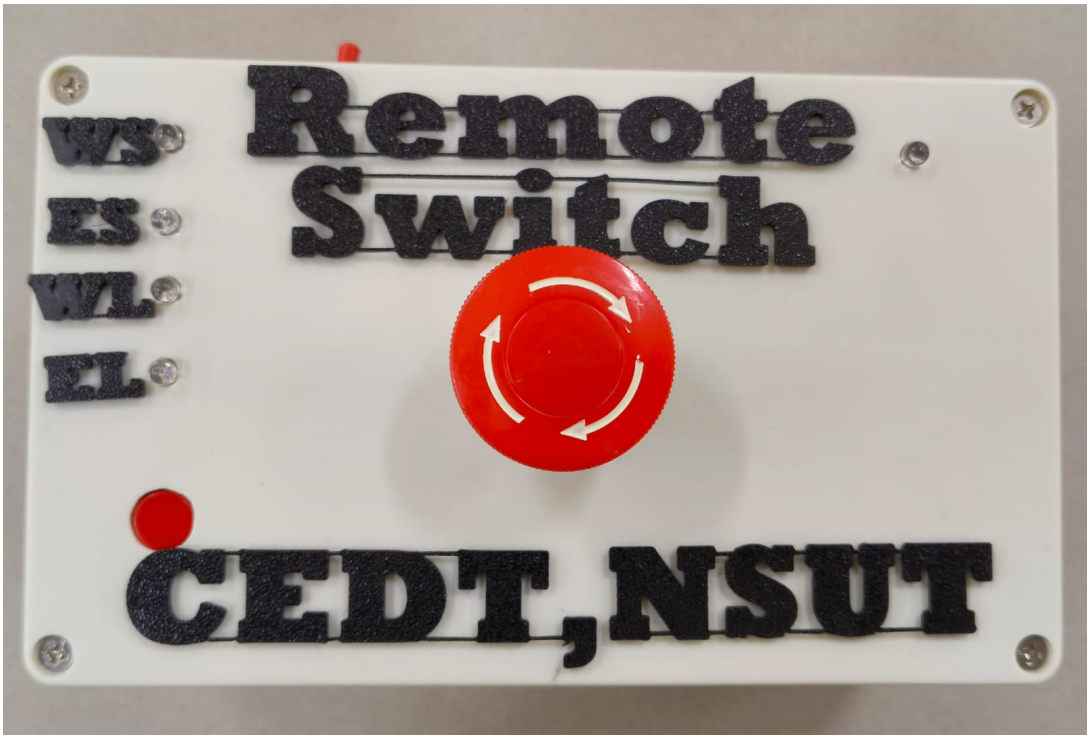


Figure 3: Front view of the Remote Switch



Figure 4: External view of the Remote Switch

Components List

- Arduino microcontroller board
- HC-05 Bluetooth module
- Emergency stop switch
- Large pushbutton switch
- SPDT power switch
- Indicator LEDs
- 220 Ω resistors (LED current limiting)
- Two 220 Ω resistors in series for power LED
- 10 kΩ pull-up resistors
- 1 MΩ and 120 kΩ resistors for battery sensing
- 1 kΩ and 2 kΩ resistors for HC-05 RX divider
- 9 V alkaline battery
- Battery connector
- Zero PCB
- Single-strand and multi-strand insulated wires
- 90-degree female Berg strip
- IP65-rated enclosure
- Hot glue, epoxy, Elfy adhesive
- Custom 3D-printed decals

Arduino Firmware

```
#include<SoftwareSerial.h>
#include <avr/io.h>
#include <avr/interrupt.h>

#define BUTTON A4
#define PushButton 9
#define BlinkingLED 3
#define VIDEO1 4
#define VIDEO2 5
#define VIDEO3 6
#define VIDEO4 7
#define lowBatteryLED 8
#define VBAT_PIN A5

volatile bool blinkEnabled = false;
volatile bool blinkState = false;

const float VREF = 1.12;
const float R1 = 1015000.0;
const float R2 = 119400.0;
const float lowBatteryVoltage = 6;

SoftwareSerial BTSerial(10, 11);

void setupTimer1ForBlink() {
  TCCR1A = 0;
  TCCR1B = 0;
  OCR1A = 31250 - 1;
  TCCR1B |= (1 << WGM12);
  TCCR1B |= (1 << CS12);
  TIMSK1 |= (1 << OCIE1A);
  sei();
}

ISR(TIMER1_COMPA_vect) {
  if (blinkEnabled) {
    blinkState = !blinkState;
    digitalWrite(BlinkingLED, blinkState);
  }
}

void setup() {
  BTSerial.begin(9600);
  analogReference(INTERNAL);

  pinMode(BUTTON, INPUT);
  pinMode(PushButton, INPUT);
  pinMode(BlinkingLED, OUTPUT);
  pinMode(VIDEO1, OUTPUT);
  pinMode(VIDEO2, OUTPUT);
  pinMode(VIDEO3, OUTPUT);
  pinMode(VIDEO4, OUTPUT);
  pinMode(lowBatteryLED, OUTPUT);

  setupTimer1ForBlink();
```

```
}

void loop() {
  static int Counter = 0;
  static bool lastPushState = HIGH;
  static bool lastButtonState = HIGH;

  bool currentPushState = digitalRead(PushButton);
  bool currentButtonState = digitalRead(BUTTON);

  int raw = analogRead(VBAT_PIN);
  float v_adc = (raw * VREF) / 1023.0;
  float v_batt = v_adc * (R1 + R2) / R2;

  if (v_batt <= lowBatteryVoltage) {
    digitalWrite(lowBatteryLED, HIGH);
    blinkEnabled = false;
    digitalWrite(BlinkingLED, LOW);
    digitalWrite(VIDEO1, LOW);
    digitalWrite(VIDEO2, LOW);
    digitalWrite(VIDEO3, LOW);
    digitalWrite(VIDEO4, LOW);
    return;
  } else {
    digitalWrite(lowBatteryLED, LOW);
  }

  if (currentButtonState == HIGH) {
    blinkEnabled = true;
  } else {
    blinkEnabled = false;
    digitalWrite(BlinkingLED, HIGH);
  }

  if (currentPushState == LOW && lastPushState == HIGH) {
    Counter = (Counter + 1) % 4;
  }
  lastPushState = currentPushState;

  digitalWrite(VIDEO1, Counter == 0);
  digitalWrite(VIDEO2, Counter == 1);
  digitalWrite(VIDEO3, Counter == 2);
  digitalWrite(VIDEO4, Counter == 3);

  if (currentButtonState == LOW && lastButtonState == HIGH) {
    BTSerial.println(Counter + 1);
  }
  lastButtonState = currentButtonState;

  delay(200);
}
```

Python Script

```
import serial
import time
import os

COM_PORT = "COM13"
BAUD_RATE = 9600

VIDEO_A = r"C:\Users\harsh\Desktop\Convocation\West Campus Part 1.mp4"
VIDEO_B = r"C:\Users\harsh\Desktop\Convocation\East Campus Part 1.mp4"
VIDEO_C = r"C:\Users\harsh\Desktop\Convocation\West Campus Part 2.mp4"
VIDEO_D = r"C:\Users\harsh\Desktop\Convocation\East Campus Part 2.mp4"

ser = serial.Serial(COM_PORT, BAUD_RATE, timeout=1)
time.sleep(2)

print("Listening for Bluetooth Serial Value...")

while True:
    if ser.in_waiting > 0:
        line = ser.readline().decode().strip()
        if not line.isdigit():
            continue

        SerialValue = int(line)
        print(f"Serial Value: {SerialValue}")

        if SerialValue == 1:
            os.startfile(VIDEO_A)
        if SerialValue == 2:
            os.startfile(VIDEO_B)
        if SerialValue == 3:
            os.startfile(VIDEO_C)
        if SerialValue == 4:
            os.startfile(VIDEO_D)
```